PPIN: Anomaly-Based Intrusion Detection on Provenance Graph via Behavior Pattern Mining

Dingwei Liu, Zhenyu Li, *Member, IEEE*, Zhibin Zhang, Zhaohua Wang, and Jin Yan

***Abstract*—Provenance graphs, which model complex relationships between system entities, are crucial for intrusion detection. While recent Provenance-based Intrusion Detection Systems (PIDSes) have shown promising results, they often struggle to capture high-level context of individual and cross behaviors, hindering the generation of *concise*, *complete*, and *intuitive* alerts for effective investigation and response. To address this gap, we introduce Ppin, an anomaly-based detection system that abstracts patterns to characterize local behaviors and correlates them over time to facilitate the reconstruction of a coherent attack narrative. Specifically, Ppin employs a memory-augmented neural network to model the relationship between processes and learnable latent categories, enabling the effective assessment of behaviors. It then introduces a pattern-based abstraction that selectively encapsulates rare entities, thereby focusing on statistical anomalies and facilitating behavior-level identification. Finally, Ppin correlates these behavior patterns based on their causal dependencies, generating alerts from highly anomalous combinations of behaviors. Extensive experiments demonstrate that Ppin outperforms state-of-the-art systems, achieving superior node-level detection performance and generating contextual alerts that effectively reveal the coherent attack narrative.**

***Index Terms*—Anomaly-based intrusion detection, data provenance, graph analysis.**

I. Introduction

MODERN organizations, including enterprises and public sector entities, face an escalating risk of intrusions targeting their critical systems [1], [2]. While these intrusions are often stealthy, they inevitably leave subtle traces within system audit logs. To capture these traces and model the complex causal relationships between system entities, researchers have widely adopted provenance graphs [3], [4], [5], [6], [7]. In these graphs, entities (e.g., processes, files, network sockets) are represented as nodes, and their interactions are represented as edges. Driven by the need to detect sophisticated attacks, such as Advanced Persistent Threats (APTs) that employ unknown techniques like zero-day exploits, recent research has focused on anomaly-based methods [8], [9], [10], [11], [12], [13], [14], [15], [16], [17]. These methods identify potential threats by detecting deviations from history. The output of these Provenance-based Intrusion Detection Systems (PIDSes) is organized into alerts, which are presented to security analysts for investigation and response. Accordingly, they typically generate alerts either as isolated anomalous nodes [10], [13], [14] or structured subgraphs [8], [11], [12].

From the perspective of a security analyst, an effective alert must present both the full picture and the critical details of an attack. To this end, a practical PIDS should satisfy three key requirements:

- *Conciseness:* The alert should include as few benign or irrelevant nodes and edges as possible, to reduce noise and the analyst's cognitive load. It is worth noting that the conciseness of *process nodes* is our main focus, as *processes* are the primary executors of attack behaviors.
- *Completeness:* The alert should include as *many* attack-related nodes and edges as possible, to represent the full attack scenario.
- *Intuitiveness:* The alert should *clearly* articulate the attack narrative, particularly the causal dependencies between different attack stages.

Nevertheless, existing approaches fail to simultaneously satisfy the three requirements. Early graph-grain methods [3], [8] ensure completeness by including all entities within an intrusion period, but this severely compromises conciseness. Recent works that generate node-level alerts [10], [13], [14], [16], [17] improve conciseness by enabling fine-grained identification, yet they lack structural intuitiveness by presenting anomalies in isolation. Some other approaches [11], [12] attempt to reconstruct the attack scenario, but they often fragment the attack context into multiple isolated alerts, each capturing only a partial view, thereby impairing intuitiveness.

These limitations stem from a fundamental gap: the absence of an attack-behavior lens in system design. While current systems often detect anomalies based on structural or temporal deviations in isolation, the multi-step attacks become evident only when viewed through a behavioral lens. Each single, localized behavior fulfills a specific *behavioral purpose*. For example, an observed behavior is a process receiving data

Received 25 November 2025; revised 1 June 2026; accepted 30 June 2026. Date of publication 3 July 2026; date of current version 1 September 2026. This work was supported by the National Natural Science Foundation of China under Grant 62502497. *(Corresponding author: Zhenyu Li.)*

Dingwei Liu is with the China Internet Network Information Center, Beijing 100070, China, and also with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100045, China (e-mail: liudingwei19g@ict.ac.cn).

Zhenyu Li and Zhibin Zhang are with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100045, China, and also with the University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: zyli@ict.ac.cn; zhangzhibin@ict.ac.cn).

Zhaohua Wang is with Computer Network Information Center, Chinese Academy of Sciences, Beijing 100045, China (e-mail: wangzh@cnic.cn).

Jin Yan is with China Internet Network Information Center, Beijing 100070, China (e-mail: yanjin@cnnic.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2026.3710034, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3710034

from an external IP and writing it to a file, consistent with the purpose of payload download. Furthermore, advanced attacks typically exhibit higher-level *behavioral coherence*, which emerges from correlating multiple, distributed behaviors with complementary purposes. For example, to establish persistence, a malicious update may involve a sequence of behaviors fulfilling distinct local purposes: downloading and then executing a payload.

This perspective offers two key design benefits. The behavioral purpose lens directs attention to suspicious behaviors. This targeted focus simplifies discrimination, enabling the precise capture of suspicious behaviors for completeness and the confident dismissal of noise for conciseness. The behavioral coherence lens then weaves these behaviors into a unified attack narrative, thereby enhancing intuitiveness.

However, integrating the perspective of attack behaviors into system design presents three fundamental challenges: ①*effective behavior assessment*, ①*attack behavior identification*, and ①*attack narrative reconstruction*. To address these challenges, we propose Ppin, an anomaly-based detection system that distills local behaviors into suspicion-aware patterns and fuses their coordinated interactions across time and space to reconstruct coherent attack contexts. At its core, Ppin introduces dedicated mechanisms designed to overcome each challenge:

1) Memory-augmented Representation Learning: A key insight is that benign processes exhibit consistent interactions with entities within specific latent categories (e.g., library files), whereas malicious processes can be signaled by atypical interactions. Ppin employs a memory module to learn and store these latent categories. This enables a process-category modeling, forming the basis for our behavior assessment.

2) Pattern-based Behavior Abstraction: Stealthy attacks often hide malicious operations among benign ones. To cut through this noise, Ppin introduces *patterns* as high-level abstractions of localized behaviors. By tracking the frequency of these patterns, Ppin quantifies their rarity, providing a powerful signal to distinguish malicious from benign behaviors.

3) Cross-Behavior Context Composition: Local context alone cannot capture behavioral coherence across attack steps. Ppin builds upon the *pattern* abstraction to link behaviors across time and space via data and control dependencies. By composing anomalous patterns into structured subgraphs, Ppin reveals how disjoint behaviors collectively form a coherent attack narrative.

We evaluate Ppin on two public datasets, DARPA TC [18] and OPTC [19], comparing against a suite of state-of-the-art PIDSes [10], [12], [13], [14]. For node-level detection, Ppin achieves a remarkable F1 score up to 61% higher than baselines, demonstrating a superior balance of precision and recall. For alert quality, Ppin delivers alerts that are 33% more precise on average than graph-based baselines, and contain a 51% higher proportion of malicious nodes within true alerts. Moreover, Ppin maintains robust process-level performance (F1>40%) on the long-running dataset with alert-level precision ranging from 50% to 100% across daily evaluations. In summary, our major contributions are:

1) We present Ppin, an anomaly-based intrusion detection system designed to generate concise, complete, and intuitive alerts on provenance graphs.
2) We introduce a memory-augmented neural network to model the interactions between processes and latent entity categories, strengthening the behavior assessment.
3) We design a pattern-based abstraction to capture local behaviors that fulfill diverse purposes. This mechanism facilitates cross-behavior analysis, linking isolated events into attack behavior coherence.
4) Our experiments demonstrate that Ppin outperforms state-of-the-art PIDSes for both node-level performance and attack reconstruction quality.
5) We publicly release an open-source version of Ppin at https://github.com/loc-l/ppin , facilitating reproducibility and further research in intrusion detection.

This paper is organized as follows. We begin with background and motivation in Section II. We then present the Ppin system, starting with an overview in Section III followed by detailed descriptions of its local behavior extraction (Section IV) and cross behavior detection (Section V) components. The system's performance is evaluated in Section VI, followed by a discussion of considerations and limitations in Section VII. We review related work in Section VIII and conclude in Section IX.

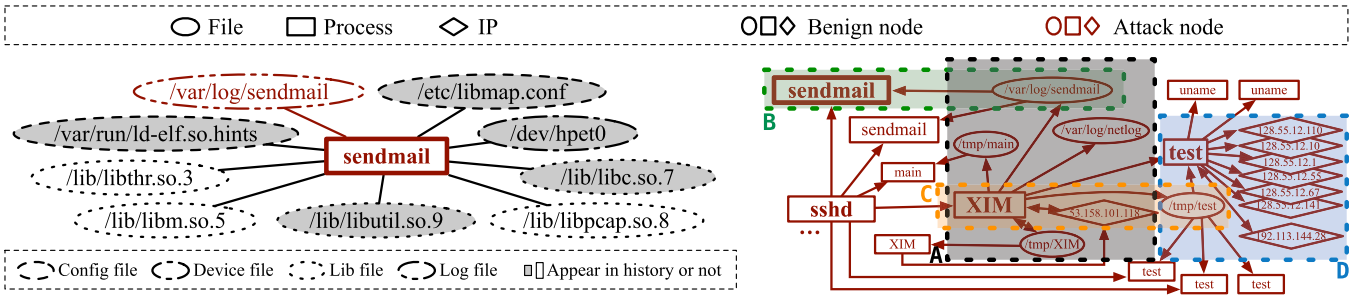
II. Background and Motivation

A. Threat Model

This work focuses on detecting potential attacks on the host through system-level audit logs, which are recorded by system-level auditing tools like Windows ETW [20] and CamFlow [21]. Our threat model is similar to previous works [8], [10], [12], [13], [14], [15]. Firstly, we assume that the collected system-level logs are complete and trustworthy, which means they contain key attack-related events. For example, the corresponding file write events in case of exploiting browser vulnerabilities to download malicious files. Secondly, attack-related structures are different from benign ones, such as the appearance of anomalous IPs and files that are not present in benign data. We generally consider events that commonly appeared in benign data as benign events. Additionally, we do not consider covert flows that are not visible in system logs, such as side channels.

B. Provenance Graph

System-level audit logs record kernel-level operations among system entities, such as processes, files, and networks. Each log entry signifies a system event, consisting of a subject (process), an object (e.g., process, file, network), and the event type (e.g., read, receive, send). A provenance graph is constructed from these auditing data, where nodes represent entities and edges represent events. Fig. 1(b) shows an example of a provenance graph, where the edge direction represents dependency flow, including data dependency (e.g., `XIM` $\xrightarrow{\text{write}}$ `/tmp/XIM`) and control dependency (e.g., `XIM` $\xrightarrow{\text{fork}}$ `test`). Notably, a provenance graph describes the entire attack procedure, including multiple long dependencies (e.g., `53.158.101.118` $\xrightarrow{\text{receive}}$ `XIM` $\xrightarrow{\text{write}}$ `/tmp/test` $\xrightarrow{\text{execute}}$ `test`) and branch-like dependencies (e.g., `XIM` $\xrightarrow{\text{fork}}$ `test` and `/tmp/test` $\xrightarrow{\text{execute}}$ `test`), where both reflect information of certain attack steps (e.g., installation,



(a) Local behavior (b) Attack example

Fig. 1. Illustration of a provenance graph from the DARPA CADETS dataset showing an attack scene. Edge directions represent data and control dependencies. (a) Fine-grained file types are distinguished by different border styles. (b) Four distinct attack behaviors are highlighted. The edge types include: `process` → `process` (fork), `process` → `file` (write/close/open/delete), `file` → `process` (read/execute), `process` → `IP` (send/close/open), and `IP` → `process` (receive).

command & control). In this way, provenance graphs capture additional relationship information and inspire the design of provenance-based intrusion detection systems (PIDSes) [3], [4], [7], [8], [10], [11], [12], [13], [14], [15], [16], [22], [23].

C. Anomaly-Based Provenance Intrusion Detection

Existing PIDSes can be roughly divided into two categories: rule-based and anomaly-based systems. Rule-based systems [4], [7], [22], [23] rely on external knowledge base to formulate rules. Their detection results rely on the completeness of the rule set, which hinders them from coping with unknown attack techniques (e.g., zero-day exploits). In contrast, anomaly-based PIDSes [3], [8], [10], [11], [12], [13], [14], [15], [16], [17] avoid dependency on attack knowledge, leveraging learning-based algorithms to model behaviors and detect anomalous ones. The intuition behind anomaly-based PIDSes is that attacks are related to behaviors distinguished from history. Existing studies define and identify behaviors at different granularities.

Early *graph-grain* methods [3], [8] abstract the host-level behavior of the entire snapshot. Consequently, they report not only malicious entities (e.g., red nodes in Fig. 1), but also *all* attack-unrelated nodes (occupying more than 99.9%) within the detected snapshot. *Node-grain* methods [10], [13], [14], [17] embed structural and temporal information of entities as the representation of entity-level behaviors to be used for downstream identification tasks. However, they still report excessive nodes, including many benign ones, failing to distinguish the attack-driven operations. For instance, ThreaTrace [10] alerts all nodes in Fig. 1(a), conflating critical attack steps with routine executions of `sendmail`. Additionally, isolated node-level alerts fail to exhibit attack context like edges in Fig. 1. Some recent PIDSes design *mixed-grain* methods [11], [12], [15] to identify anomalies at fine-grain and then construct structural alerts. Nevertheless, their results lack intuitiveness, especially regarding the coherence of attack behavior. Taking Kairos [12] as an example, a specific attack is fragmented into multiple alerts, thereby overlooking the dependencies between contained behaviors.

The limitations of existing methods underscore a critical need to design detection systems oriented to attack behaviors. Specifically, prioritizing behaviors fulfilling malicious purposes enables the precise retention of attack details, while reconstructing behavior correlations facilitates an intuitive, multi-stage attack coherence.

D. Motivations and Challenges

To motivate our system design, consider an attack scenario from the CADETS dataset (DARPA TC [18]). This attack is carried out through a series of *coherent behaviors*, each fulfilling a distinct *behavioral purpose*, such as downloading a malicious implant, attempting privilege escalation, and scanning for open ports. Fig. 1 visualizes part of the provenance graph: one subfigure shows a single example of local behavior, while the other reveals the broader attack narrative.

However, while the attack's logic is transparent when viewed through the lens of attack behaviors, transforming raw system traces into alerts that are *concise*, *complete*, and *intuitive* remains challenging.

C1. How to accurately assess the anomalousness of behaviors fulfilling diverse purposes? Behaviors can fulfill either benign or malicious purposes. Anomaly-based methods typically assess anomalousness by quantifying deviation from history. We observe that system entities belong to latent, fine-grained categories (e.g., library files), and each type of benign process maintains a stable relationship with these categories, consistent with its behavioral purposes. For example, in Fig. 1, three benign, previously unseen neighbors (e.g., `/lib/libthr.so.3`) can be categorized as library files, similar to the historically seen `/lib/libc.so.7`. In contrast, the attack-related file `/var/log/sendmail` appears to belong to a previously unseen log type, reflecting the anomalous execution of a log file. Nevertheless, such latent categories are difficult to label and vary across different scenarios (e.g., database files only exist on hosts running databases).

C2. How to identify suspicious behaviors fulfilling malicious purposes? Here, malicious purposes are those that facilitate an attack. A compromised process typically interacts with nodes

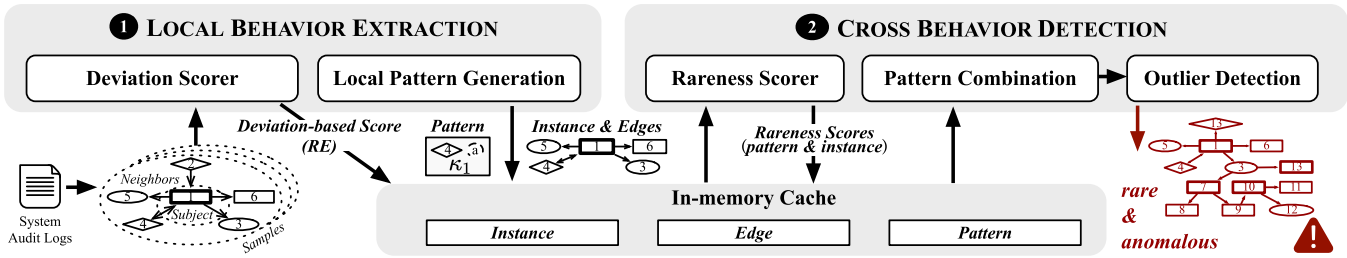


Fig. 2. The system architecture of Ppin.

fulfilling malicious purposes alongside numerous benign ones, whose sheer number can dominate the local neighborhood. For instance, in Fig. 1, `sendmail` accesses one malicious node and eight benign neighbors. A naive approach that reports all neighbors leads to excessive false positives at the node level (e.g., the eight benign nodes in the example). At the behavior level, this noise can obscure attack behaviors, causing them to be conflated with similar benign activities and resulting in missed detections. Therefore, it is crucial to capture the key anomalies that fulfill malicious purposes, enabling the system to distinguish malicious behaviors from unseen but benign ones.

C3. How to uncover cooperative behaviors that reveal behavioral coherence? Typically, an entire attack is orchestrated by multiple processes, each executing distinct behaviors. Fig. 1(b) illustrates four representative attack behaviors within this scenario: (A) The attacker exploits a pre-privileged process, `XIM`, to download micro-APT implants from `53.158.101.118`. These implants are disguised as temporary or log files (e.g., `/var/log/sendmail`). (B) The attacker executes privilege escalation for one implant via `sendmail` executing `/var/log/sendmail`. (C) In a later time window, `XIM` downloads another implant, `/tmp/test`. (D) The attacker conducts network reconnaissance via port scans, illustrated when the process `test` accesses six internal IPs (e.g., `128.55.12.110`). These interdependent behaviors collectively constitute the coherent behaviors of initial compromise and foothold establishment. However, neither existing snapshot-level modeling nor fine-grained approaches (e.g., node- or edge-level analysis) can effectively capture such cross-behavior collaboration.

To address these challenges, we propose Ppin, a novel framework that abstracts behavior patterns that characterize behavioral purposes and links anomalous patterns into behavioral coherence. To tackle *C1*, Ppin employs a learnable memory to incorporate latent category information, thereby enhancing the model's discriminative power. To address *C2*, Ppin introduces a pattern-based abstraction that captures the rare yet meaningful portions of a behavior, effectively encapsulating local behaviors while enabling precise, behavior-level identification. To resolve *C3*, Ppin performs cross-behavior detection over these local patterns, breaking free from temporal and spatial locality to reconstruct the behavioral collaboration.

III. Overview

We first provide a high-level overview of Ppin, an anomaly-based intrusion detection system as shown in Fig. 2. Later, we detail two major components of Ppin: ❶ *Local Behavior Extraction* (Section IV), ❷ *Cross Behavior Detection* (Section V).

1) Local Behavior Extraction: The local behavior extraction (detailed in Section IV) extracts local anomalous information of observed behaviors in each time window (15 minutes as default). It consists of two key components: *deviation scorer* and *local pattern generation*. To precisely assess the anomalousness of certain behaviors (*C1*), *deviation scorer* uses a learnable memory to incorporate latent category information, essentially modeling the process-category relationships. To reduce the interference of benign noise on anomaly identification (*C2*), *local pattern generation* focuses only on historically rare portions to extract patterns. A process's local behavior is expressed as a local pattern with a deviation score (RE in Fig. 2) and is cached in memory for further analysis.

2) Cross Behavior Detection: The cross behavior detection (detailed in Section V) periodically generates structured alerts from the in-memory cache. It operates in three steps. First, *rareness scorer* evaluates the collective rareness of patterns to distinguish suspicious behaviors from common benign ones. Combined with the deviation score, Ppin filters out low-risk processes (*C2*). Second, *pattern combination* correlates patterns with interdependencies (e.g., shared nodes) to construct behavior combinations. Finally, *outlier detection* assesses the anomalousness of these combinations by their constituent behavior patterns. The combinations with outlier anomalousness are reported as alerts exhibiting coherent attack narratives (*C3*).

IV. Local Behavior Extraction

The local behavior extraction is composed of two components, as shown in Fig. 3. The *Deviation Scorer* module captures the relationship between subject (process) and its local behavior by assessing the anomaly score based on deviation from history. The *Local Pattern Generation* module filters out benign noise within local behaviors to focus on rare portions, generating expressive local patterns. Thereafter, we detail the in-memory cache to store the outputs of this module.

A. Deviation Scorer

Intuitively, the attack occurs in processes whose local behaviors deviate from history. To capture this deviation, we can learn the relationship between the process (subject) and its related behavior as shown in Fig. 4(a). Delving deeper into the process-local behavior pairs, we consider that this relationship can be represented as an association between the process and certain latent nodes as shown in Fig. 4(c). For example, PSQL-generated

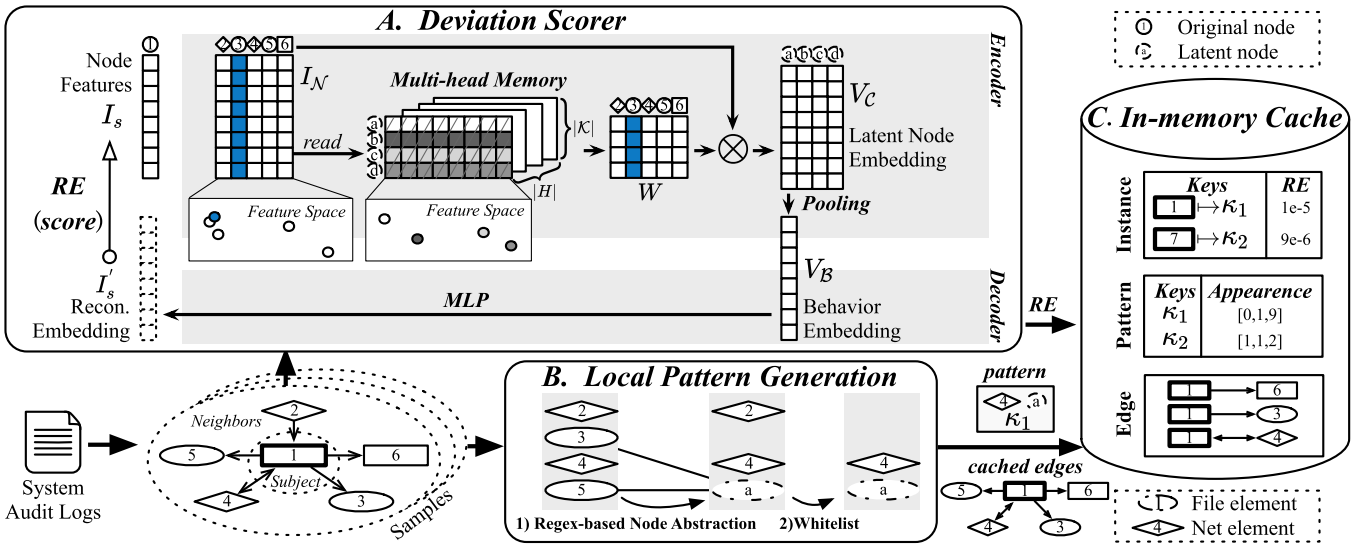


Fig. 3. The Local Behavior Extraction overview.

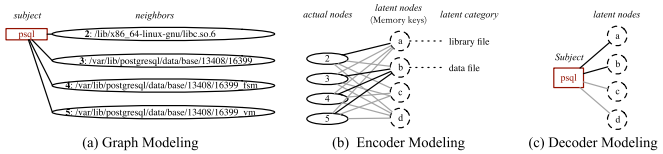


Fig. 4. An illustration of relationship modeling of MEMAN. Black and gray edges represent high and low weights.

processes typically read and write data files (ⓑ in Fig. 4) and library files (ⓐ in Fig. 4). Since these files are automatically generated by PSQL, they exhibit highly similar features and thus can be mapped to a shared latent node in the feature space. That is, there exists a weighted relationship between system nodes and latent nodes. Therefore, we design MEMAN to learn latent categories (Fig. 4(b)) and model the relationship between subject and the latent categories (Fig. 4(c)).

As shown in Fig. 3, MEMAN leverages a learnable memory to generate behavior embeddings, where each memory key is viewed as a clustering centroid (i.e., latent node like ⓐ). This memory stores latent category information learned from history. Overall, MEMAN deploys an encoder-decoder architecture [24] that uses behavior embeddings to reconstruct the corresponding subject features. This essentially establishes the relationship between subject and latent categories as Fig. 4. We next describe MEMAN in detail.

1) Preprocess: In the context of provenance graphs, we treat a subject (process) and its direct neighbors within a specific time window as a *local sample* (see Fig. 3). Each local sample satisfies locality in both time and space for a certain process. Since the direct neighbors of a subject (process) indicate its current anomalous status, we formally define the local behavior of a process p in a time window w as follows:

Definition 1: For a subject (process) p in time window w, the local behavior of p is $\mathcal{B}(p,w) = \{v_o | v_o \in \mathcal{V}_{\text{tt}} obj, (p, v_o) \in \mathcal{E}_w\}$, where $\mathcal{E}_w$ is the edge set of w.

The *deviation scorer* takes local samples (i.e., subject and local behavior pairs) to assess anomalousness by deviation from history. The initial features of system nodes are derived from their attributes using feature hashing [25] (also used by [12]), enabling rapid and space-saving conversion of string features into vectors. Table I lists all entity types and their attributes considered.

TABLE I
ELEMENTS CONSIDERED BY PPIN

Element	Type	Attributes
Node	Process	PID, Name
	File	Name, Path
	IP	IP address
Edge	clone, fork, execute, load, creat, read, write, open, close, delete, connect, accept, open, close, receive, send	

2) Architecture of MEMAN: Let $I_s \in \mathbb{R}^d$ and $I_{\mathcal{N}} \in \mathbb{R}^{|\mathcal{N}| \times d}$ denote the attribute features of the subject and its neighbors, where d is the feature dimension. MEMAN encodes the neighbor feature $I_{\mathcal{N}}$ into a behavior embedding $V_{\mathcal{B}}$ via a memory-augmented encoder, then decodes $V_{\mathcal{B}}$ to reconstruct the subject feature I_s.

Encoder: We employ an array-structured memory like [26], consisting of several learnable keys $\mathcal{K}$, which serve as soft clustering centroids (*a.k.a.* latent categories) for the input space (Fig. 3(a)). These memory keys are organized as a multi-head array, where different heads can project input into diverse subspaces. The multi-head mechanism allows for capturing features from multiple perspectives simultaneously [27] and facilitates joint attention across different representation subspaces [28]. In our context, legitimate and malicious rarities that are indistinguishable in one subspace could exhibit significant differentiation in another.

The memory reading treats each node feature as a query $q \in I_{\mathcal{N}}$. A soft assignment weight $W \in \mathbb{R}^{|\mathcal{N}| \times |\mathcal{K}|}$ is computed by normalizing the similarity between these queries and the memory keys. The embedding for the latent nodes $V_c \in \mathbb{R}^{|\mathcal{K}| \times d}$ is then obtained by a weighted aggregation over the queries as $W^\top \times I_{\mathcal{N}}$. Each row of V_c represents a learned latent node, and

this matrix is subsequently pooled to generate the behavior embedding $V_{\mathcal{B}}$. Unlike static mapping, the soft weight ensures that the subtle semantic signatures are encoded rather than erased. Note that we mask the subject in the encoder and embed only its neighbors. This design prevents the subject's own features from being exposed to the decoder, ensuring that the reconstruction relies solely on the behavioral embedding.

Decoder: MEMAN's decoder (see Fig. 3(a)) uses a two-layer MLP to reconstruct the node features I_s of the corresponding subject. That is, the decoder reconstructs each subject from its local behavior, which has been expressed as features of latent nodes. Let I'_s denote the reconstructed feature. Then, Ppin uses MSELoss [29] to measure the reconstruction error, which is minimized during training:

$$RE = \text{MSELoss}(I'_s, I_s) \tag{1}$$

In this way, MEMAN essentially learns the relationship between subject and latent nodes as Fig. 4(c).

3) Training and Testing: MEMAN learns by both reconstruction error RE and unsupervised clustering loss. Unsupervised clustering loss is defined by Kullback-Leibler (KL) divergence between soft assignments and a clustering-friendly auxiliary distribution to train memory keys. In particular, to ensure the stability of training, memory keys are updated less frequently than the rest of the model weights (see [26] for more details on training).

At test time, Ppin views reconstruction error RE as anomalousness scored by MEMAN. MEMAN tends to score higher RE for processes that have unseen local behaviors, but lower for those with historically seen behaviors. This deviation-based assessment will be used to identify highly anomalous processes in the *Cross Behavior Detection*.

B. Local Pattern Generation

The *local pattern generation* (see Fig. 3(b)) aims to filter out benign noise in local behaviors, thus abstracting local behaviors into patterns that focus on the relatively anomalous parts. Patterns consist of merged elements, each mapping to similar entities (e.g., files named like `/tmp/security.xxxx`). With a previously generated element whitelist, Ppin removes common elements and preserves only rare and potentially anomalous ones. The formal definition of a pattern is as follows:

Definition 2: For a subject (process) p in time window w, the pattern, $\kappa(p, w)$, is extracted from p's local behavior $\mathcal{B}(p, w)$, $\kappa(p, w) = \{e_o = g(f(v_o)) | v_o \in \mathcal{B}(p, w) \setminus \mathcal{V}_{\text{tt}} proc\}$, where $f(\cdot)$ is regex-based node abstraction, $g(\cdot)$ is whitelist, e_o represents an element in κ corresponding to the object v_o.

Note that patterns only include `file` and `IP` as elements. Since processes with similar attributes can play very different roles, a single process element cannot reflect and distinguish this difference and is highly likely to cause confusion. In detail, the generation of patterns consists of two steps.

1) Regex-Based Node Abstraction $f(\cdot)$: Ppin maps similar file entities to the same element to enhance patterns' generalization. Taking a common system-generated temporary file type as an example, `/tmp/security.rLNvx11 J` and `/tmp/security.ooxmTtEN` can be expressed by the same expression `/tmp/security.xxxx`[1]. This can be represented as the mapping from ③ and ⑤ to ⓔ in Fig. 3(b). In this way, an element can indicate several to thousands of similar files. We implement it by matching regular expressions, mapping similar files into the same regular expression.

2) Whitelist $g(\cdot)$: In order to filter out the noise of benign elements within local information, Ppin takes the historical rareness of elements into consideration. Like many previous works [12], [30], [31], Ppin considers an element that frequently appears in benign logs less likely to be anomalous. Ppin automatically extracts a whitelist from historical data based on two principles. An element is considered benign if (i) appears in most historical windows. This is to filter elements accessed by long-running background tasks in the host; (ii) is accessed by a vast number of processes. This is to filter commonly shared elements (e.g., library files). Note that we additionally follow the established security practices (e.g., Microsoft Defender for Endpoint [32]) and explicitly exclude sensitive or critical system files (e.g., `/device/harddiskvolume1/windows/service/profiles/networkservice/ntuser.dat`). PPIN excludes elements matching the whitelist (e.g., ② in Fig. 3(b)) when extracting local patterns.

The resulting patterns, which capture clues for malicious purposes, are cached to support downstream detection and the reconstruction of coherent attack narratives.

C. In-Memory Cache

In order to record the occurrence of similar behaviors, Ppin defines *hit* to describe a process p that behaves similarly to a pattern κ, expressed as $p \mapsto \kappa$. We formally define pattern hit ($\mapsto$) as follows:

Definition 3: For a subject (process) p in time window w, p's local behavior pattern is denoted as $\kappa(p, w)$. Suppose K is all recorded patterns in memory, if $\kappa \in K$ and κ is similar to $\kappa(p, w)$, then $p \mapsto \kappa$.

Since each pattern is represented as a set of elements, Ppin evaluates the similarity between two patterns based on the proportion of their intersection. A similarity threshold τ_s is applied to identify meaningful matches, with its impact evaluated in Section VI-F.

1) Cached Pattern Update: For any pattern $\kappa(p, w)$ generated by process p in window w, if there exists a pattern κ in memory that $p \mapsto \kappa$, then Ppin updates κ by adding elements in $\kappa(p, w)$. Otherwise, Ppin stores $\kappa(p, w)$ in memory as a newly hit pattern. For ease of explanation, we use κ to denote the pattern that p hits.

2) Cache Organization: The stored data is organized into three parts as shown in Fig. 3(c): (i) pattern table, which stores behavior patterns and their occurrence statistics (e.g., occurrence time windows as *Appearance* in Fig. 3(c)). (ii) instance table,

[1]The core regex set is directly reusable across datasets derived from the same OS and audit source. In practice, deployment-specific changes are lightweight additions (e.g., application log prefixes) rather than re-tuning from scratch.

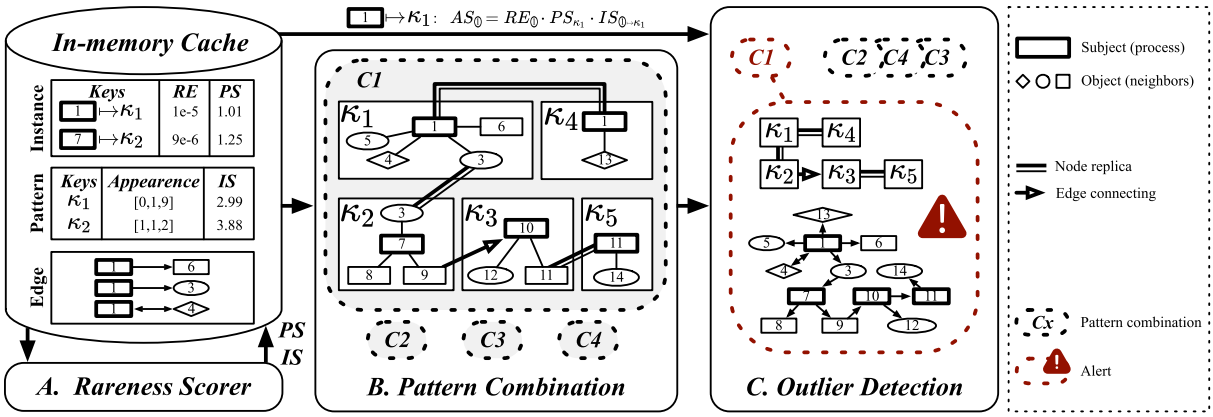


Fig. 5. Design overview of the Cross Behavior Detection.

which stores processes' local behaviors and their anomalousness. Each instance is a process and behavior pair, denoted as $p \mapsto \kappa$. This table maps each instance to its deviation-based score RE by the *Deviation Scorer* (Section IV-A). (iii) edge table, which stores anomalous edges corresponding to generated patterns in the current window. Ppin updates all three parts after each time window.

D. Complexity Analysis

Let V and E denote the number of distinct nodes and edges observed during the detection period (24 hours by default). Let $V_{\max}$ and $E_{\max}$ denote the max number of nodes and edges per window. MEMAN first transforms node features using the memory-augmented module with the complexity of $\mathcal{O}(V_{\max})$, followed by neighbor-wide pooling at cost $\mathcal{O}(E_{\max})$.

V. Cross Behavior Detection

Periodically, Ppin analyzes the in-memory cache to find *rare* and *anomalous* behavior combinations and reconstructs behavioral coherence via structural relationships. We next detail the components of *Cross Behavior Detection* as shown in Fig. 5.

A. Rareness Scorer

This module captures two kinds of runtime rareness, aiming to distinguish anomalous behaviors from frequently appearing new benign ones. The pattern-level score measures the rareness of different behavior patterns, while the instance-level score considers the subtle difference among different pattern instances (*a.k.a.* subject and its neighbors). We leverage the idea of inverse document frequency (IDF) [33] to define these two scores.

1) Pattern-Level Score: In fact, attackers will try their best to hide attack clues. In the time dimension, the attack process tends not to continue the same attack pattern for a long time. In other words, attack patterns tend to appear in much fewer time windows than benign ones. In the quantitative dimension, attackers tend to minimize the execution of each attack behavior pattern to avoid exposure. That is, attack patterns tend to appear fewer times than benign ones. Therefore, Ppin defines the pattern-level rareness score PS for κ as:

$$PS_\kappa = \min\left(\ln\left(\frac{|W|+1}{H_k^{Window}}\right), \ln\left(\frac{\max_\kappa(H_\kappa^{Pattern})+1}{H_\kappa^{Pattern}}\right)\right) \tag{2}$$

where $|W|$ denotes the total number of windows passed through, $H_\kappa^{Pattern}$ denotes the hit number for a pattern κ, H_κ^{Window} denotes the number of windows containing κ (i.e., the size of *Appearance* in Fig. 5).

2) Instance-Level Score: There are situations where the difference between benign and attack behavior is so subtle that they hit the same pattern. For example, an injected process accesses benign nodes and attack-related ones at the same time, while another attribute-similar process only accesses these benign nodes. In this case, only the former instance hitting anomalous elements is attack. For a process p within a time window w, considering its current pattern $\kappa(p,w)$ and the associated hit pattern κ, if an element e exists at the intersection of $\kappa(p,w)$ and κ, we refer to it as an element hit of e. In particular, benign and attack elements show variance in hit distributions. Benign elements tend to be hit more times, while attack ones are hit fewer. Therefore, we define the instance-level rareness score IS for $p \mapsto \kappa$ to assess such element difference.

$$IS_{p\mapsto\kappa} = \frac{1}{|\kappa \cap \mathcal{N}(p)|} \sum_{e\in\kappa\wedge e\in\mathcal{N}(p)} \frac{H_\kappa^{Pattern}+1}{H_e^{Element}} \tag{3}$$

where $\mathcal{N}(p)$ denotes the set of p's neighbors, e denotes element in both κ and $\mathcal{N}(p)$, $H_e^{Element}$ denotes the hit number of element e. p hit κ inherently ensures $|\kappa \cap \mathcal{N}(p)| > 0$.

Recall that Ppin uses the *deviation scorer* (see Section IV-A) to assess deviation-based score RE for each instance ($p \mapsto \kappa$). Ppin combines it with the above two scores as:

$$AS_{p\mapsto\kappa} = RE_{p\mapsto\kappa}^{\gamma} \times PS_\kappa \times IS_{p\mapsto\kappa} \tag{4}$$

$AS_{p\mapsto\kappa}$ denotes the final score of $p \mapsto \kappa$. The sensitivity bias γ is a design choice that controls the relative emphasis between deviation-based anomaly scoring and rarity-based assessment, allowing Ppin to adapt to different deployment scenarios. Considering that a process may have several patterns representing

behaviors in different time windows (i.e., attack steps), an average is used to evaluate the overall anomalousness.

$$AS_p = \frac{1}{|K_p|} \sum_{\kappa \in K_p} (AS_{p \mapsto \kappa}) \tag{5}$$

where K_p denotes all patterns that process p hits.

Ppin marks the processes with high AS_p exceeding a percentile threshold β as candidates, with the impact of β evaluated in Section VI-F. In this way, Ppin prioritizes processes whose behaviors *deviate from history* and are *rare*.

Although Kairos [12] also employs IDF to measure rareness, our novelty lies in its specific granularity and application. While Kairos utilizes IDF for node-level filtering, our approach employs IDF for pattern-level and instance-level scoring to quantify behavior-level rareness. Furthermore, instead of binary filtering, we combine these IDF-based scores with deviation-based scores for a joint decision, avoiding hard cutoffs based on a single dimension.

B. Pattern Combination

As discussed in Section II-D, a complete attack involves multiple interdependent anomalous behaviors that collectively exhibit behavioral coherence. In Ppin, pattern instances are abstracted as isolated subgraphs with subject(S)-to-object(O) structures. Topological correlation between two subgraphs happens only when there are either shared nodes or cross-pattern edges. Accordingly, Ppin identifies two types of topological dependencies between behavior patterns:

1) *Node Replica*, intersecting via shared nodes. Depending on the shared node's roles, this yields three sub-cases: S-S (shared as Subject in both, e.g., κ_1 and κ_4 in Fig. 5(b)), O-O (shared as Object in both, e.g., κ_1 and κ_2 in Fig. 5(b)), and S-O/O-S (shared across Subject and Object roles, e.g., κ_3 and κ_5 in Fig. 5(b));
2) *Edge Connection*, linked by cross-pattern edges. Since typical edges between processes (S) and files/IPs (O) are internal to pattern instances in Ppin, only inter-process edges (e.g., FORK) bridge distinct patterns (e.g., κ_2 and κ_3 in Fig. 5(b)).

According to the dependencies above, Ppin generates pattern combinations from hit patterns of identified candidates. Each combination includes one to several closely related behaviors. The original nodes (system entities) and edges (events) are restored together with combinations. We next describe how Ppin detects attacks and constructs alerts from these pattern combinations.

C. Outlier Detection

Ppin assesses combination anomalousness using the average of included patterns.[2] For each pattern, Ppin picks one instance's anomalousness $AS_{p \mapsto \kappa}$. Then the formula for calculating the anomalousness of combination c is:

$$AS_c = \frac{1}{|K_c|} \sum_{\kappa \in K_c} (AS_{p \mapsto \kappa}) \tag{6}$$

where K_c denotes all patterns included in c. Note that in a combination, Ppin considers only one instance (*a.k.a.* one process) for each pattern. This is to avoid certain repeated patterns playing too much influence on the overall anomalousness[3]. In this way, Ppin scores each combination by its included behavior patterns.

Ppin leverages the Modified Z-Score [34] for adaptive alerting based on test data, avoiding the need for predefined fixed thresholds. We select the Modified Z-Score for outlier detection because of its robustness to extreme values and skewed distributions [35], a property that aligns well with the combination score distribution. Specifically, Ppin computes M, the median of the scores $\{AS_c\}$ and MAD, the median of the absolute deviations $\{|AS_c - M|\}$. The Modified Z-Score for each combination c is then defined as:

$$MZ_c = \frac{0.6745(AS_c - M)}{MAD} \tag{7}$$

Combinations with MZ_c exceeding a threshold τ_{mz} are flagged. The threshold is a tunable parameter that balances sensitivity and specificity, with its impact evaluated in Section IV-E5 of the supplementary material.

To depict the entire attack scene, Ppin constructs alert graphs from restored nodes and edges of identified combinations. In this way, Ppin's alerts present contextual attack information including long dependencies (e.g., ① $\xrightarrow{\text{write}}$ ③ $\xrightarrow{\text{read}}$ ⑬ in Fig. 5(c)) and branch-like dependencies (e.g., ③ $\xrightarrow{\text{read}}$ ⑬ and ③ $\xrightarrow{\text{read}}$ ⑦ in Fig. 5(c)). Also, Ppin can exhibit coherent attack narratives by behavior interdependencies.

D. Complexity Analysis

The pattern combination is performed on a filtered subgraph containing at most βV nodes (default $\beta = 0.3$) and αE edges ($0.001 \leq \alpha \leq 0.007$ measured empirically), yielding a complexity of $\mathcal{O}(\beta V + \alpha E)$. The resulting subgraphs are significantly smaller than the original, due to the noise filtering mechanism and the removal of structures associated with low-risk processes in prior components.

VI. Evaluation

We implemented all the components of Ppin in Python. MEMAN in *Anomaly Scorer* (see Section IV-A) is implemented with PyG [36] with default model setups where the node feature dimension of I_s and $I_{\mathcal{N}}$ is 16, the memory key number $|\mathcal{K}| = 10$, and the memory head number $|H| = 3$. The sensitivity bias γ is set to 1 by default and adjusted to 2 when higher deviation sensitivity is beneficial. For example, $\gamma = 2$ applies to Windows-derived traces, where the proliferation of auto-generated file path variants introduces noise into rarity-based statistics. γ is

[2]Note that the scope of these combinations aligns with the in-memory cache's retention period (defaulting to 24 h).

[3]Empirical validation on CADETS demonstrates that use of one instance per pattern outperforms the use of all instances. See Section IV-E9 of the supplementary material for detailed ablation results.

TABLE II
DATASET SPLIT FOR DARPA TC TRAINING AND TESTING, WHERE **BOLD** DAYS CONTAIN ATTACKS

Dataset	Train Data	Test Data
CADETS	2 / 3 / 4 / 5	**6** / 7 / 8 / 9 / 10 / **11** / **12** / **13**
THEIA	3 / 4 / 5 / 9	**10** / **11** / **12**
TRACE	2 / 3 / 4 / 5 / 6	**13**
Five Directions	4 / 5 / 6 / 7 / 8	**9**

evaluated in Section IV-E11 of the supplementary material. We run all experiments on a server with Intel(R) Xeon(R) Silver 4214 CPU (126 GB) and an NVIDIA GeForce RTX 3080 Ti GPU (12 GB). In the following, we first present the experimental setup for evaluation, then introduce the results in detail. We mainly focus on the following aspects:

A1: The comparison with state-of-the-art detectors that alert at node-level (Section VI-B1) and graph-level (Section VI-B2).

A2: The performance on a longer period of time, including node-level performance (*conciseness* and *completeness*) and alert-level attack restoration (*intuitiveness*) (Section VI-C).

A3: The robustness against mimicry attacks (Section VI-D).

A4: The overhead study (Section VI-E).

A5: The effect of key components and hyperparameter study (Section VI-F).

A. Experimental Setup

1) Datasets: We evaluate Ppin with the publicly available DARPA TC [18] and OPTC [19] datasets, which have been widely used for performance evaluation in existing literature [8], [10], [11], [12], [13], [14]. These datasets were collected by the DARPA Transparent Computing program, which recorded audit logs of red team hosts during an adversarial engagement. The blue team attacked the red team's hosts multiple times and left attack-related events in the red team's audit logs (such as file read/write/execute, network receive/send). Table II exhibits the train/test split of DARPA TC datasets, where only the data before the attack is used for training. The dates with attacks are **bolded**. Note that we evaluate Ppin on several consecutive days on CADETS and THEIA, rather than a short period (less than one day) as in previous works [10], [13], [14]. This is to evaluate Ppin's capability to keep false positives at a low level when running continuously online. Additionally, we offer a comparison to baselines on TRACE and Five Directions.

For evaluation on OPTC, we follow the same three attack scenarios as FLASH [13], including (i) *PowerShell Empire* on host 201, (ii) *Data Leakage* on host 501, and (iii) *Malicious Upgrade* on host 051. This setup allows us to assess Ppin under diverse adversarial behaviors, ranging from initial compromise to data exfiltration and persistence. The training data range is the same as that of FLASH, containing only audit logs that temporally precede the test data.

Across all datasets, whitelists are generated using only training data to avoid information leakage from the test set.

2) Labeling: We refer to the groundtruth provided by DARPA [18] and label the critical `Process`, `File`, and `IP` that constitute the attack graph. We have aligned our node-level labels with the open-source ones [37]. For more details about the labeling results, please refer to Section I in the supplementary material.

3) Metrics: We use metrics at both node-level and alert-level for performance evaluation. Alert-level metrics are designed for PIDSes with structural alerts. Note that ↑ is to signify larger is better, while ↓ is to signify smaller is better in following statistics.

Node-level: Let TP_n, FP_n, and FN_n denote the number of true positives, false positives, and false negatives at node level. True/false indicates whether a node is marked abnormal/normal by the detector, while positive/negative indicates whether a node is attack/benign in the label. Node-level precision is computed by $\frac{\text{TP}_n}{\text{TP}_n+\text{FP}_n}$. Node-level recall is computed by $\frac{\text{TP}_n}{\text{TP}_n+\text{FN}_n}$. Node-level F1 is computed by $2 \times \frac{\text{Precision}_n \times \text{Recall}_n}{\text{Precision}_n+\text{Recall}_n}$. The above metrics are evaluated for each node type respectively. Here, precision and recall respectively reflect *conciseness* and *completeness*, with process precision being the primary measure of conciseness since processes are the central subjects of behaviors. High recall is required to ensure the completeness of attack information, while high accuracy is essential to ensure attack information is not drowned in vast irrelevant structures. Only with both can experts truly avoid time-consuming backtracking and investigation when reviewing alerts. Therefore, we employ F1 to evaluate this.

Alert-level: We quantify *intuitiveness* along three dimensions. Let TP_A, FP_A, and FN_A denote the number of true positives, false positives, and false negatives of reported graph-structured alerts. (i) *Alert-level precision* is defined as $\frac{\text{TP}_A}{\text{TP}_A+\text{FP}_A}$. (ii) *Alert number for each case* equals TP_A, reflecting the extent of inter-behavior connections. (iii) *Average proportion of attack nodes in attack alerts* is computed by $\frac{1}{|\mathcal{A}_{\text{att}}|}\sum_{A_i \in \mathcal{A}_{\text{att}}} \frac{|V_{A_i} \cap V_{\text{att}}|}{|V_{A_i}|}$, where $\mathcal{A}_{\text{att}}$ is the set of alerts containing at least one groundtruth attack node (i.e., those contributing to TP_A), V_{A_i} is the node set of alert A_i and V_{att} is the set of groundtruth attack nodes. A higher proportion indicates that alerts are more focused on actual attacks and contain fewer irrelevant nodes, thereby reducing cognitive load for sysadmins during review.

4) Baselines: We choose four state-of-the-art open-source PIDSes as baselines: ThreaTrace [10], FLASH [13], MAGIC [14], and Kairos [12]. ThreaTrace [10], FLASH [13], and MAGIC [14] are node-level detectors using graph learning and have verified their performance over previous anomalous log detectors. Kairos [12] is a mixed-grain method, which emphasizes the importance of understanding intrusion and outputs tight suspicious alerts. Since all methods involve multiple hyperparameters that can affect performance, we used their code and models released in comparative experiments.

B. Comparison With State-of-The-Art PIDSes

1) Comparison With Node-Level Detectors: In this section, Ppin is compared to three baselines: ThreaTrace [10], FLASH [13], and MAGIC [14]. They perform node-level detection and report anomalous nodes as alerts. To ensure consistency in comparison standards, we reprocess baselines' outputs by

TABLE III
COMPARISON BETWEEN THREATRACE [10], FLASH [13], MAGIC [14], AND PPIN. PREC IS SHORT FOR PRECISION. REC IS SHORT FOR RECALL

		Process						File						IP					
		tp↑	fp↓	fn↓	*rec*↑	*prec*↑	F1↑	tp↑	fp↓	fn↓	*rec*↑	*prec*↑	F1↑	tp↑	fp↓	fn↓	*rec*↑	*prec*↑	F1↑
DARPA TC CADETS	ThreaTrace	10	54	8	0.56	0.16	0.24	7	83	1	0.88	0.08	0.14	7	5	4	0.64	0.58	0.61
	FLASH	13	17	5	0.72	**0.43**	0.54	7	55	1	0.88	0.11	0.20	7	3	4	0.64	0.70	0.67
	MAGIC	4	141	14	0.22	0.03	0.05	1	90	7	0.12	0.01	0.02	7	5	4	0.64	0.58	0.61
	PPIN	18	25	0	**1.00**	0.42	**0.59**	8	21	0	**1.00**	**0.28**	**0.43**	10	1	1	**0.91**	**0.91**	**0.91**
DARPA TC THEIA	ThreaTrace	5	8	57	0.08	0.38	0.13	1	812	3	0.25	1e-3	2e-3	2	4	3	0.40	0.33	0.36
	FLASH	22	1	40	0.35	**0.96**	0.52	4	952	0	**1.00**	4e-3	8e-3	5	14	0	**1.00**	0.26	0.42
	MAGIC	60	146	2	**0.97**	0.29	0.45	4	69	0	**1.00**	0.05	0.10	5	4	0	**1.00**	0.56	0.71
	PPIN	57	36	5	0.92	0.61	**0.74**	4	65	0	**1.00**	**0.06**	**0.11**	5	0	0	**1.00**	**1.00**	**1.00**
DARPA TC TRACE	ThreaTrace	14	1930	29	**0.33**	0.01	0.01	3	2074	0	**1.00**	1e-3	3e-3	10	7	0	**1.00**	0.59	0.74
	FLASH	10	814	33	0.23	0.01	0.02	2	239	1	0.67	0.01	0.02	10	55	0	**1.00**	0.15	0.27
	PPIN	13	22	30	0.30	**0.37**	**0.33**	3	48	0	**1.00**	**0.06**	**0.11**	10	22	0	**1.00**	**0.31**	**0.48**
DARPA TC Five Directions	ThreaTrace	3	32	35	0.08	0.09	0.08	2	78	8	0.20	0.03	0.04	0	0	2	-	-	-
	FLASH	3	330	35	0.08	0.01	0.02	0	23	10	-	-	-	0	0	2	-	-	-
	PPIN	29	25	9	**0.76**	**0.54**	**0.63**	8	158	2	**0.80**	**0.05**	**0.09**	2	1	0	**1.00**	**0.67**	**0.80**
OPTC-201 Powershell Empire	FLASH	9	51	11	**0.45**	0.15	0.23	2	9	3	0.40	**0.18**	**0.25**	0	0	8	-	-	-
	PPIN	6	9	14	0.30	**0.40**	**0.34**	4	46	1	**0.80**	0.08	0.15	4	14	4	**0.50**	**0.22**	**0.31**
OPTC-501 Data Leakage	FLASH	14	26	16	0.47	0.35	0.40	2	58	7	0.22	0.03	0.06	3	0	0	**1.00**	**1.00**	**1.00**
	PPIN	17	27	13	**0.57**	**0.39**	**0.46**	8	146	1	**0.89**	**0.05**	**0.10**	3	38	0	**1.00**	0.07	0.14
OPTC-051 Malicious Update	FLASH	2	13	81	0.02	0.13	0.04	0	42	4	-	-	-	2	3	2	**0.50**	**0.40**	**0.44**
	PPIN	19	19	64	**0.23**	**0.50**	**0.31**	3	87	1	**0.75**	**0.03**	**0.06**	2	15	2	**0.50**	0.12	0.19

filtering out attribute-free entities and recompute their node-level metrics on different entity types.

Table III shows the performance comparison between Ppin and baselines. On DARPA TC, Ppin achieves the best F1, demonstrating Ppin's ability to simultaneously achieve high precision and recall. Taking THEIA as an example, we observe that ThreaTrace and FLASH report fewer FPs for `process`, resulting in high precision, but at the cost of a significant number of FNs (57, 40 *vs.* 5 by Ppin), which leads to low recall. While MAGIC achieves slightly higher recall than Ppin, it raises many more FPs than Ppin (146 *vs.* 34). For `file` and `IP`, all baselines exhibit lower precision than Ppin. Specifically, ThreaTrace detects only a small part of the attack; it misses most processes and key object nodes like malicious IPs (e.g., `161.116.88.72`, `46.153.68.151`) and files (e.g., `/var/log/wdev`). Considering FPs, ThreaTrace generates a lot on files, including vast temporary files (e.g., 773 files like `/run/shm/org.chromium/xxxx`). Although FLASH detects all anomalous files and IPs, it misses many attack processes, including all writes to anomalous file `/tmp/memtrace.so`. Similar to ThreaTrace, FLASH reports a lot of temporary files (e.g., 188 files like `/proc/xxxx/cmdline`). In contrast, missed nodes by Ppin only include processes performing no attack yet or only forking attack processes. As for MAGIC, the large number of FPs on `process` hinders understanding of attack information. After tracing with the nodes detected by MAGIC, we generate an attack alert with only 24% of nodes that are attack-related (*vs.* 84% in Ppin).

On OPTC, Ppin achieves the highest F1 for `process`across all three test scenarios, demonstrating superior capability in capturing process-centric attack behaviors. For `IP`and `file`, Ppin achieves better recall. For example, Ppin identifies critical IPs (e.g., `142.20.56.202`for powershell empire stager download on *PowerShell Empire*) and files (e.g., the leaked `export.zip`on *Data Leakage*), which are missed by FLASH. Additionally, since all node-level baselines' outputs are isolated nodes, sysadmins still need extra traceback to restore attack context.

Additionally, a degradation in `file`/`IP`precision is observed across both Ppin and baselines in specific settings. The root cause is the one-to-many topology of provenance graphs, where a single subject process naturally interacts with numerous passive resources (files/IPs). This is exacerbated by strictly labeling only files/IPs directly related to groundtruth. For example, in the Data Leakage scenario (OPTC Host 501), the attack process `chrome.exe` is correctly detected by Ppin. However, its reported neighbors include 15 benign files alongside 1 malicious file. A similar phenomenon occurs with IPs. This leads to low file/IP precision, despite accurate process detection.

2) Comparison With Detector Generating Alert Graphs: Kairos [12] streamingly processes logs in each window and periodically performs a detection algorithm to report summarized alert graphs. Kairos' goal is to construct a complete but concise attack story, which is consistent with Ppin. For consistency, we reanalyze the outputs of Kairos by merging network nodes with the same IP. We notice that Kairos merges processes with the same name (e.g., `firefox`processes with different PIDs). Actually, these processes may behave differently. For instance, two processes both named `firefox`play different roles: the malicious one accesses `141.43.176.203`, while benign instances do not. Therefore, we report only the results on node-level metrics of `file`and `IP` in Table IV. Thereafter, we compare the quality of attack restoration of Ppin and Kairos.

TABLE IV
COMPARISON RESULTS BETWEEN PPIN AND KAIROS [12]. *PREC* IS SHORT FOR PRECISION

			File						IP					
		day	tp↑	fp↓	fn↓	*recall*↑	*prec*↑	F1↑	tp↑	fp↓	fn↓	*recall*↑	*prec*↑	F1↑
CADETS	Kairos	6	3	615	0	**1.00**	5e-3	0.01	5	19	0	**1.00**	0.21	0.34
	PPIN	6	3	2	0	**1.00**	**0.60**	**0.75**	5	1	0	**1.00**	**0.83**	**0.91**
THEIA	Kairos	10	0	1719	3	-	-	-	3	24	1	**0.75**	0.11	0.19
	PPIN	10	3	6	0	**1.00**	**0.33**	**0.50**	2	0	2	0.50	**1.00**	**0.67**
	Kairos	11	0	0	0	-	-	-	0	0	2	0.00	0.00	0.00
	PPIN	11	0	1	0	-	-	-	2	0	0	**1.00**	**1.00**	**1.00**
	Kairos	12	3	1843	1	0.75	2e-3	3e-3	4	49	1	0.80	0.08	0.14
	PPIN	12	4	65	0	**1.00**	**0.06**	**0.11**	5	0	0	**1.00**	**1.00**	**1.00**

TABLE V
ALERT-LEVEL COMPARISON BETWEEN PPIN AND KAIROS. ♯A REPRESENTS THE NUMBER OF ALERTS THAT HIT THE ATTACK LABEL. ♯B REPRESENTS THE NUMBER OF BENIGN ALERTS. *PRECISION* DENOTES ALERT-LEVEL PRECISION. *AVE.* PROP. DENOTES THE AVERAGE PROPORTION OF ATTACK NODES IN ATTACK ALERTS.

		CADETS	THEIA		
		6	10	11	12
♯*A*↓ / ♯B↓	Kairos	2 / 10	**2** / 8	0 / 0	4 / 9
	PPIN	**1 / 0**	**2 / 1**	**1 / 0**	**1 / 1**
Precision↑	Kairos	0.17	0.20	0.00	0.31
	PPIN	**1.00**	**0.67**	**1.00**	**0.50**
Ave. prop.↑	Kairos	0.25	0.05	0.00	0.40
	PPIN	**0.67**	**0.71**	**0.93**	**0.85**

Node-level: Kairos performs worse in precision (4× to 86× lower than Ppin). This is due to the many more FPs that Kairos reports, especially on `file`[4]. Note that Kairos misses the attack scene on THEIA (11th), where `profile (deleted)` sent data to `161.116.88.72` and `7.149.198.40`. Such omission is possibly due to Kairos' design. It uses the sum of anomalousness of multiple overlapped time windows to detect outlier window sequences. This may lead Kairos to generate an alarm only when the attack has occurred for a period across multiple windows. However, the duration of the missed attack on the 11th is 30 minutes (2 time windows).

Alert-level: (i) alert-level precision. As shown in Table V, Ppin significantly outperforms Kairos (> 0.50*vs.* < 0.31) on alert-level precision, generating fewer alerts (< 3*vs.* > 8) per day. *(ii) Alert number for each case.* Kairos describes each attack scene by 2 to 4 separate alerts, each containing only a small part of the attack. This means sysadmins need to piece them back together to recover the full attack information. Ppin, in contrast, maintains most dependencies in one single alert, clearly presenting attack information. (iii) *Average proportion of attack nodes in attack alerts.* As shown in Table V, Ppin's true alerts are more information-dense with up to 66% higher average proportion of attack nodes. In summary, although Kairos successfully identifies attack periods in most cases, it performs worse than Ppin on attack narrative recovery.

[4]To ensure consistency of comparison, the files reported in Table IV distinguish between different names, while Kairos adopts an additional simplification to reduce node size in the output graphs. After such simplification, the file number reported by Kairos on THEIA 10th and 12th is reduced to 142 and 191 (*vs.* 9 and 69in Ppin)

C. Performance of Ppin over a Long Period

To evaluate Ppin's ability when running continuously online, we test it on several consecutive days and report alerts daily (the same detection period as Kairos [12]) on DARPA TC. To clearly show the daily performance, we split the results according to dates. As shown in Table VI, Ppin exhibits excellent node-level performance in terms of both recall (*completeness*) and accuracy (*conciseness*). Specifically, the average recall and precision of `process` is 0.98 and 0.61, benefiting from Ppin's ability to make full use of both latent category information from history and runtime rareness information at pattern level. As to the alert-level performance, Table VII reports the metrics on the proposed three dimensions. Ppin achieves a high *alert-level precision* with up to one false alert per day, while *alert number for each case* for Ppin is 1in most cases (6 out of 7). Besides, the *average proportion of attack nodes in attack alerts* is over 43%. All the above results indicate Ppin's ability to generate *intuitive* alerts. Additionally, detailed case studies are provided in the supplementary material Section IV-A.

D. Ppin against Mimicry Attacks

We further evaluate Ppin's robustness against mimicry attacks. Following [38], we randomly inject benign common structures into Theia 12th. Table VIII reports the affected node-level metrics of `process`. Specifically, under the default threshold β for high-risk candidate selection, Ppin missed only one more process, whose suspicious behavior is captured by other detected processes (receiving from `161.116.88.72`). That is, MEMANretains sufficient information by anomaly scores for further judgement. With a slightly looser $\beta = 0.4$, Ppin achieves consistent FNs with that on original datasets at a cost of 2 more FPs. Additionally, under both settings of β, attack nodes occupy over 94% in attack alerts, maintaining good *intuitiveness* for sysadmins to review. These results demonstrate Ppin's robustness against mimicry attacks. Details of data simulation and case study are provided in the supplementary material Section IV-B.

TABLE VI
PPIN'S NODE-LEVEL PERFORMANCE. *REC* IS SHORT FOR RECALL. *PREC* IS SHORT FOR PRECISION. UNLISTED DATES CONTAIN NO ALERTS

		Process						File						IP					
	day	tp↑	fp↓	fn↓	*rec*↑	*prec*↑	F1↑	tp↑	fp↓	fn↓	*rec*↑	*prec*↑	F1↑	tp↑	fp↓	fn↓	*rec*↑	*prec*↑	F1↑
CADETS (6th - 13th)	**6**	2	2	0	1.00	0.50	0.67	3	2	0	1.00	0.60	0.75	5	1	0	1.00	0.83	0.91
	11	1	3	0	1.00	0.25	0.40	2	4	1	0.67	0.33	0.44	3	1	0	1.00	0.75	0.86
	12	18	25	0	1.00	0.42	0.59	8	21	0	1.00	0.28	0.43	10	1	1	0.91	0.91	0.91
	13	4	4	0	1.00	0.50	0.67	6	7	0	1.00	0.46	0.63	5	1	0	1.00	0.83	0.91
THEIA (10th - 12th)	**10**	46	1	3	0.94	0.98	0.96	3	6	0	1.00	0.33	0.50	2	0	2	0.50	1.00	0.67
	11	12	0	0	1.00	1.00	1.00	0	1	0	-	-	-	2	0	0	1.00	1.00	1.00
	12	60	36	5	0.92	0.62	0.75	4	65	0	1.00	0.06	0.11	5	0	0	1.00	1.00	1.00

TABLE VII
ALERT-LEVEL PERFORMANCE OF PPIN. ♯A REPRESENTS THE NUMBER OF ALERTS THAT HIT THE ATTACK LABEL. ♯B REPRESENTS THE NUMBER OF BENIGN ALERTS. *PRECISION* DENOTES ALERT-LEVEL PRECISION. *AVE.*PROP. DENOTES THE AVERAGE PROPORTION OF ATTACK NODES IN ATTACK ALERTS. PPIN REPORTS NO ALERTS ON UNLISTED DATES.

	CADETS				THEIA		
day	**6**	**11**	**12**	**13**	**10**	**11**	**12**
♯*A* ↓ / ♯B↓	1/0	1/0	1/1	1/1	2/1	1/0	1/1
Precision↑	1.00	1.00	0.50	0.50	0.67	1.00	0.50
Ave. prop.↑	0.67	0.43	0.44	0.62	0.71	0.93	0.85

TABLE VIII
RESULTS OF PPIN ON MIMICRY DATASETS

	Process					
	tp↑	fp↓	fn↓	*rec*↑	*prec*↑	F1↑
Original ($\beta = 0.3$)	60	36	5	0.92	0.62	0.75
Mimicry ($\beta = 0.3$)	59	30	6	0.91	0.66	0.77
Mimicry ($\beta = 0.4$)	60	38	5	0.92	0.61	0.74

TABLE IX
TIME BREAKDOWN REPORT ON CADETS. *LBE* DENOTES LOCAL BEHAVIOR EXTRACTION; *CBD* DENOTES CROSS BEHAVIOR DETECTION; *AV* DENOTES ALERT VISUALIZATION.

Day	*LBE* (s)					*CBD* (s)	*AV* (s)
	Min	Median	90^{th}	Max	Total		
6	0.002	0.37	0.59	7.97	51.82	0.029	0.11
7	0.271	0.39	0.70	8.69	57.03	0.036	-
8	0.283	0.37	0.63	8.71	52.07	0.033	-
9	0.240	0.31	0.43	9.13	40.06	0.020	-
10	0.233	0.30	0.45	8.80	49.07	0.026	-
11	0.002	0.31	0.65	9.19	49.26	0.034	0.10
12	0.217	0.32	0.51	9.59	47.75	0.032	0.31
13	0.232	0.32	0.53	1.00	25.88	0.017	0.19

E. Overhead Study

Building on the performance evaluation in Section VI-C, we assess Ppin's overhead considering time (long-running) and workload variance in this section. Note that evaluation is conducted on a server concurrently running other experimental tasks, with the CPU utilization of these co-existing tasks fluctuating between 25% and 35%.

1) Time: Table IX presents the statistical information of time breakdown on CADETS. Specifically, Ppin processes each window in at most 9.59 s, significantly lower than the 15-minute window duration. With local processing pipelined with window data generation, the end-to-end latency for daily detection is around 1 s. Such low computational overhead benefits from Ppin's design, where the final graph operations are conducted on only a small high-suspicion subgraph derived from the edge cache (e.g., containing 628 to 1,305 edges on CADETS). Besides, we estimate the peak cache memory footprint to be below 40 MB during the evaluated days (see details in the supplementary material Section IV-C). Taken together, Ppin guarantees consistently low end-to-end overhead and controlled memory usage over an extended period.

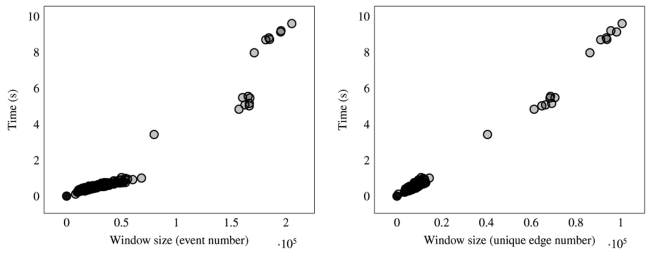


Fig. 6. Processing time per window when varying window size.

2) Workload: As shown in Fig. 6, per-window processing time scales primarily with the number of events and unique edges. Specifically, it exhibits an approximately linear relationship with the number of unique edges. The system maintains low latency (around 1 s) for typical workloads (up to 100,000 raw events or 50,000 unique edges). Even under the dataset's peak workload with ∼200,000 raw events, the maximum latency remains below 10s. This demonstrates Ppin's time efficiency across diverse event densities within fixed-length windows.

F. Ablation Study

This section presents independent effect analyses of Ppin's key components, followed by a hyperparameter study to discuss their influence on performance.

1) Effect of MEMAN: We evaluate the effectiveness of the neural network design of Ppin, especially the memory module. We compare the memory-augmented encoder with other two types: SAGE (used in ThreadTrace [10] and FLASH [13]) and GAT (used in MAGIC [14]). In detail, the discrimination model MEMAN in *deviation scorer*(Section IV-A) is modified by respectively replacing the encoder with two-layer SAGE and GAT. Both models are trained with the same data as MEMAN. We test their performance on CADETS (12th), leaving

TABLE X
COMPARISON RESULTS WITH DIFFERENT ENCODER TYPES ON CADETS 12TH. MEM IS FOR MEMORY-AUGMENTED ENCODER, WHICH IS USED IN PPIN. *PREC* IS SHORT FOR PRECISION.

Type	Process						File						IP					
	tp↑	fp↓	fn↓	*recall*↑	*prec*↑	F1↑	tp↑	fp↓	fn↓	*recall*↑	*prec*↑	F1↑	tp↑	fp↓	fn↓	*recall*↑	*prec*↑	F1↑
SAGE	5	6	13	0.28	**0.45**	0.34	8	16	0	1.00	**0.33**	**0.50**	5	1	6	0.45	0.83	0.59
GAT	4	10	14	0.22	0.29	0.25	8	63	0	1.00	0.11	0.20	6	2	5	0.55	0.75	0.63
MEM	18	25	0	**1.00**	0.42	**0.59**	8	21	0	**1.00**	0.28	0.43	10	1	1	**0.91**	**0.91**	**0.91**

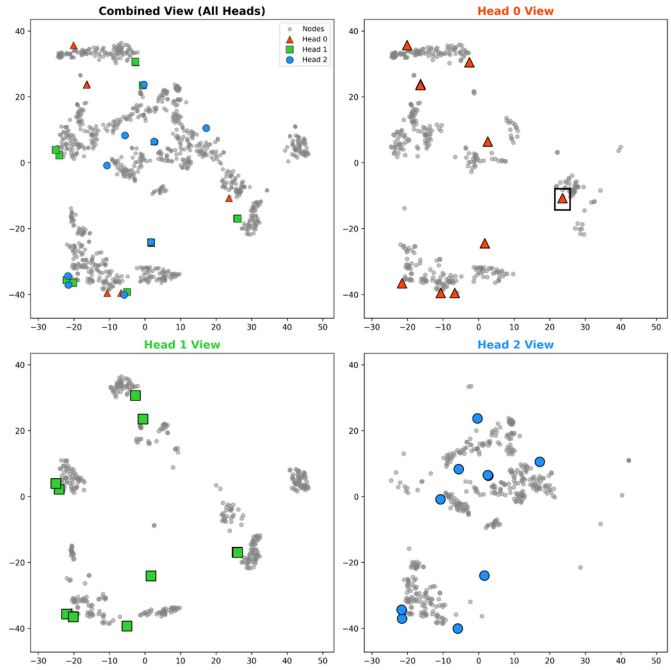
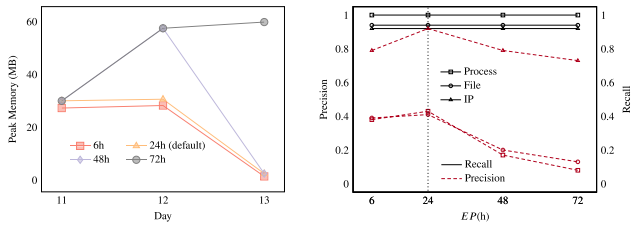


Fig. 7. Latent space visualization of regarding memory keys and close node queries on THEIA.



(a) Peak memory usage. (b) Performance.

Fig. 8. Impact of eviction period EPon in-memory cache and detection performance across CADETS 11th-13th.

the remaining components of Ppin unchanged. As shown in Table X, compared with memory-augmented encoder used by Ppin, changing the encoder type to SAGE and GAT results in fewer TPs and more FNs on `process`and `IP`, leading to worse recall. In contrast, MEMANconsiders latent category information, which enhances its expressive power. This is consistent with our discussion in Section II-D. Besides, we evaluate the effectiveness of multi-head memory. The default 3-head model comprehensively outperforms that with single-head memory on node-level metrics (please refer to Table III in the supplementary material Section IV-E6).

We have conducted a detailed investigation into the latent information captured by the memory keys. Specifically, we retrieved the top-50 nearest queries (from the training data nodes) for each key and projected them, along with the memory keys, into a 2D latent space using t-SNE. We provide both a combined view containing all three attention heads and head-specific views. As shown in Fig. 7, different attention heads focus on distinct regions and capture information from distinct perspectives. Furthermore, localized overlaps between keys from different heads in the combined view suggest cross-head consensus on critical system contexts. Crucially, we observe that certain memory keys exhibit strong alignment with human-interpretable semantics. For instance, regarding the memory key highlighted by the box in Head 0 (Fig. 7), the majority of its nearest neighbors are identified as Firefox cache entries and Chromium shared memory. This suggests that this key has learned a coherent latent category related to browser transient storage.

2) Effect of Rareness Scorer: Ppin's final assessment for processes (see Section V-A) includes three parts: deviation-based score (RE), pattern-level score (PS), and instance-level score (IS). We conduct an ablation on CADETS 12th to specify their contributions. Table XI shows that using any one or two of IS, RE, and PS in isolation leads to a significant number of false positives. In contrast, their combination simultaneously enforces assessments on anomalousness, behavioral rarity, and instance rarity, greatly improving detection precision.

3) Effect of Eviction Strategy: By default, Ppin employs periodical eviction with the eviction period EPequivalent to detection period DP=24h. For comparison, we evaluate Ppin under more frequent strategy (EP=DP=6h) and less frequent ones ($EP \in \{48, 72\}$h, DP=24h) on the multi-day attack scenario CADETS (11th-13th). As shown in Fig. 8(b), recall remains stable across all eviction strategies, indicating that even under aggressive eviction Ppin does not miss critical attack-related history. However, Ppin achieves the best precision under the default setting (EP=DP=24h), with longer cache leading to degradation due to extra induced benign behaviors. Moreover, Fig. 8(a) suggests a limited impact on cache growth when $EP \leq 24$h, while longer EPleads to sublinear cache expansion across days. In deployment, EP can be configured as a tunable parameter according to operational requirements, such as detection latency and memory constraints.

4) Effect of Hyperparameter: Specifically, we evaluate performance under different detection periods DP, similarity threshold τ_s, candidate threshold β, and time window length L. Here, we report the results for CADETS and include results on

TABLE XI
COMPARISON RESULTS WITH DIFFERENT COMBINATIONS OF IDF SCORES ON CADETS 12TH. $RE \times PS \times IS$ INDICATES THE RESULT OF THE COMPLETE VERSION OF PPIN. *PREC.* IS SHORT FOR PRECISION.

Score Combination	Process						File						IP					
	tp↑	fp↓	fn↓	recall↑	prec.↑	F1↑	tp↑	fp↓	fn↓	recall↑	prec.↑	F1↑	tp↑	fp↓	fn↓	recall↑	prec.↑	F1↑
RE	17	39	1	0.94	0.30	0.46	8	8	0	1.00	0.50	0.67	10	1	1	0.91	0.91	0.91
PS	0	26	18	0.00	0.00	0.00	0	8	8	0.00	0.00	0.00	0	5	11	0.00	0.00	0.00
IS	3	155	15	0.17	0.02	0.03	8	55	0	1.00	0.13	0.23	10	4	1	0.91	0.71	0.80
$RE \times PS$	15	33	3	0.83	0.31	0.45	8	45	0	1.00	0.15	0.26	5	2	6	0.45	0.71	0.56
$RE \times IS$	9	119	9	0.50	0.07	0.12	8	44	0	1.00	0.15	0.27	10	1	1	0.91	0.91	0.91
$PS \times IS$	18	120	0	1.00	0.13	0.23	8	96	0	1.00	0.08	0.14	10	14	1	0.91	0.42	0.57
$RE \times PS \times IS$	18	25	0	1.00	0.42	**0.59**	8	21	0	1.00	0.28	**0.43**	10	1	1	0.91	0.91	**0.91**

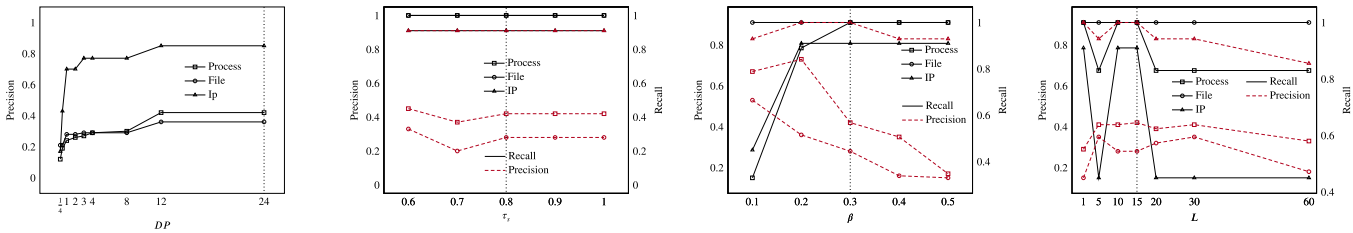


Fig. 9. Detection performance on CADETS (12th) while varying specific hyperparameter.

DARPA TC in the supplementary material Section IV-E due to space constraints. In brief, a small DPhurts precision when lacking sufficient runtime statistics. An extremely small τ_s causes similar patterns to be mixed, hindering accurate pattern-level rareness capture. Given that the sensitivity curve is largely flat above 0.8, τ_s is a low-sensitivity parameter and 0.8 serves as a safe default. β is related to the trade-off between precision and recall. Ldominates the grain of pattern extraction and affects the trade-off between over-fragmentation and over-generalization. Besides, Lhas minimal impact on peak cache usage. As shown in Fig. 9 and the supplementary material, our default settings yield strong performance across evaluated datasets. In practice, we recommend using the default settings as starting points and adjusting them within observed stable ranges based on deployment-specific characteristics, such as latency tolerance and expected alert volume.

VII. DISCUSSION AND LIMITATION

A. Evasion and Data Poisoning

Attackers may use advanced techniques to hinder PIDSes from detecting the hints of an attack. As validated in Section VI-D, Ppin demonstrates robustness against mimicry attacks (e.g., [39]). Since Ppin extracts patterns at node level and considers subtle node differences, behaviors with slight difference compared to history can still be recognized. A remaining risk is attackers poisoning the training dataset, which can be mitigated by techniques [40], [41], [42] on tamperproof and defense.

B. Concept Drift

Host execution environments naturally evolve over time, introducing emerging benign behaviors absent from historical data. This phenomenon, known as *concept drift*, makes it challenging for models trained solely on historical data to accurately classify these novel behaviors. To mitigate this, Ppin incorporates three targeted mechanisms: the memory-augmented model design that captures stable latent category information, enhancing robustness against surface-level variations in entity features; runtime rareness scoring that evaluates behavior rareness, which helps to suppress new frequent noise characteristic of benign drift; and dynamic thresholding via a distribution-based outlier detection mechanism, which adapts decision boundaries to the current data distribution. Even if unseen but benign elements are falsely reported alongside novel behaviors, they will be isolated into separate alert graphs due to their lack of structural connections to attacks, preserving the clarity of true attack traces. Empirically, models trained on CADETS (Days 2-5) maintained stable performance over the subsequent eight days (Days 6-13), demonstrating resilience to natural concept drift over this period. Furthermore, Ppin can leverage alarm volume as an indicator to detect significant concept drift. To ensure sustained adaptability, administrators can apply direct retraining strategies [12], [14], or adopt more sophisticated approaches like Chaos [17], which balances prior knowledge retention with new behavior adaptation.

C. Noise Reduction

Since neural network models base decisions on historical training data, they may exhibit limited generalization to previously unseen system behaviors, potentially leading to increased false positives on unseen but benign activity. Fortunately, like prior work [12], [30], [31], [43], we adopt a frequency-based whitelisting strategy for noise reduction from the view of resource nodes (e.g., files).

To mitigate the risk of potential over-permissiveness and under-filtering, we further incorporate established security practices (e.g., Microsoft Defender for Endpoint [32]) and explicitly exclude sensitive or critical system files (e.g., `/device/harddiskvolume1/windows/serviceprofiles/networkservice/ntuser.dat`) from the whitelist. This conservative design balances effectiveness and practicality in noise reduction. While the current implementation represents one viable configuration, we acknowledge that stronger security guarantees could be achieved through more fine-grained policies, such as operation-level filtering or provenance-aware access control, which we identify as valuable avenues for future refinement.

D. Temporal Sparsity in Long-Duration Attacks

In extreme stealth scenarios, where attack clues are highly sparse over weeks or months and fall below detection thresholds within individual cycles, relying solely on in-memory caching may be insufficient. We acknowledge the value of augmenting the system with persistent, disk-based cache dumps to enable retrospective correlation across extended temporal spans. We consider developing detection methods over such long-term histories as a promising direction for future work, complementary to the current in-memory framework.

E. Cache Surge From Benign Noise

In our current experiments, cache memory consumption is easily manageable for standard host memory due to Ppin's time window design. For example, processing 8 days of CADETS logs (∼25 GB raw data) results in a peak cache consumption of only ∼30 MB. In cases where cache usage surges due to a large volume of benign noise generated between attack steps, adopting hierarchical storage is a standard practice for scaling up detection systems [15], [44].

F. Practical Usage Concerns

Real-world production logs may exhibit higher event volume and more ambiguous benign/malicious boundaries than those we used in this paper. Nevertheless, Ppin processes streaming logs in fixed time windows, which bounds the per-window graph size and prevents unbounded memory growth. Our overhead analysis in Section VI-E also confirms that per-window processing time scales approximately linearly with the number of unique edges. We also considered the possible impacts of ambiguous benign/malicious boundaries in Ppin's design choices, including a memory-augmented deviation scorer that distinguishes malicious deviations from benign variations (Section IV-A); rareness estimation at both pattern and instance levels to reduce sensitivity to noisy nodes/edges (Section V-A); and intrusion detection based on behavior combinations to suppress false alerts from sporadic benign fluctuations (Section V-B). The evaluation in Section VI-D also confirms Ppin's robustness under mimicry attacks that intentionally blur benign/malicious boundaries. We have released Ppin's open-source implementation for further validation.

VIII. Related Work

Intrusion Detection: Besides the anomaly-based PIDSes mentioned in Section II, a prominent line of work employs attack-knowledge-driven rules for intrusion detection. Sleuth [4] proposes a tag propagation algorithm that alerts low-confidence nodes accessing highly sensitive nodes. Morse [22] follows this tag-based direction and introduces tag decay to further reduce false positives. To better understand intrusion, Holmes [23] maps provenance data to specific attack stages based on expert knowledge, and Poirot [7] constructs attack query graphs to match abnormal structures in provenance graphs. Captain [45] advances the ability to automatically adapt to diverse execution environments. Yet, these systems generally require expert knowledge to construct detection rules (e.g., tag distribution rules and attack query graphs), limiting their generalizability to unknown threats.

Distinct from these rule-based systems, semi-supervised and supervised learning techniques have also been explored for PIDSes. For instance, Slot [46] utilizes graph reinforcement learning for adaptive neighbor selection, while Vodka [47] employs knowledge distillation to transfer detection capabilities from a complex teacher model to a lightweight student model. However, they rely on labeled attack data for model training.

In contrast, Ppin and other anomaly-based PIDSes identify attacks based on deviations from historical norms, independent of prior attack data or attack knowledge. Within the anomaly-based domain, we differentiate our work from recent studies [48], [49] that focus on precise attack entity identification. Instead, our approach emphasizes the reconstruction of coherent attack narratives, offering an alternative perspective for intrusion detection.

Another line of research detects intrusions using system logs without provenance graph modeling [50], [51], [52], [53]. DeepLog [50], LogRobust [52], and LogGAN [53] view system logs as natural language sequences and utilize Long Short-Term Memory (LSTM) based models to learn normal patterns. Log2vec [51] converts logs into heterogeneous graphs and performs graph embedding to detect anomalies via clustering. ST-Graph [54] constructs workflow graphs from distributed system logs, fusing semantic, spatial, and temporal features to enable GCNN-based graph-level anomaly detection. These solutions are complementary to our provenance graph-based approach.

In the broader context of threat detection, various graph optimizations have been explored, such as attribute-based graph construction for IoT IDS [55] and behavioral feature modeling for activity graphs [56]. Concurrently, while primarily applied in general graph learning, Neural Architecture Search (NAS) for GNNs [57], [58], [59] represents a promising avenue for autonomously optimizing model structures, potentially enhancing the adaptability and efficiency of GNN-based detection systems.

Intrusion Investigation: In addition to detection, existing works facilitate attack investigation and understanding based on system provenance. Watson [60] abstracts and clusters semantically similar behaviors to assist analyst investigation. DEPCOMM [61] and DEPIMPACT [62] reconstruct the attack scene from a limited set of point-of-interest (POI), detected by

PIDSes such as [48], [49]. Since Ppin reconstructs coherent attack narratives, it does not rely on such separate post-investigation.

Memory-augmented Neural networks: Memory-augmented neural networks (MANNs) utilize diverse external memory components, enabling the retrieval of information. This explicit access to experiences significantly enhances the model's expressive capability. MANNs have wide applications across multiple domains. In question answering, studies such as [63], [64], [65], [66] have integrated memory with other potent neural network modules like CNNs [67] and BERT [68]. In biomedical research, [26] employs array-structured memory to perform graph coarsening. In the field of recommendation systems, works such as [69] integrate memory when addressing multi-domain recommendation challenges. To the best of our knowledge, Ppin is the first work to incorporate a MANN into provenance-based intrusion detection.

IX. Conclusion

In this paper, we propose Ppin, an anomaly-based intrusion detection system that generates *concise*, *complete*, and *intuitive*alerts to support rapid review and response by sysadmins. To enable effective behavior assessment, Ppin employs memory-augmented representation learning to enhance the discriminative power for local behaviors. For precise identification of malicious behaviors, Ppin introduces a pattern-based abstraction that encapsulates rare behavior portions, providing strong signals for behavior-level detection. To reconstruct attack narratives, Ppin correlates anomalous patterns through data and control dependencies, composing them into structured alert graphs. Our experiments show that Ppin outperforms state-of-the-art PIDSes at node-level metrics and contextual quality of alerts.

References

[1] J. Navarro, A. Deruyver, and P. Parrend, "A systematic survey on multi-step attack detection," *Comput. Secur.*, vol. 76, pp. 214–249, 2018.

[2] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1851–1877, 2nd Quart. 2019.

[3] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA: Association for Computing Machinery, 2016, pp. 1035–1044.

[4] M. N. Hossain et al., "SLEUTH: Real-time attack scenario reconstruction from cots audit data," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 487–504.

[5] P. Gao, X. Xiao, Z. Li, F. Xu, S. R. Kulkarni, and P. Mittal, "AIQL: Enabling efficient attack investigation from system monitoring data," in *Proc. USENIX Annu. Tech. Conf.*, 2018, pp. 113–126.

[6] P. Gao et al., "SAQL: A stream-based query system for real-time abnormal system behavior detection," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 639–656.

[7] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, "POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1795–1812.

[8] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *Proc. Netw. Distrib. System Secur. Symp.*, 2020.

[9] Q. Wang et al., "You are what you do: Hunting stealthy malware via data provenance analysis," in *Proc. 27th Annu. Netw. Distrib. System Secur. Symp.*, 2020.

[10] S. Wang et al., "THREATRACE: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3972–3987, 2022.

[11] F. Yang, J. Xu, C. Xiong, Z. Li, and K. Zhang, "PROGRAPHER: An anomaly detection system based on provenance graph embedding," in *Proc. 32nd USENIX Secur. Symposium. USENIX Assoc.*, 2023, pp. 4355–4372.

[12] Z. Cheng et al., "Kairos: Practical intrusion detection and investigation using whole-system provenance," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 3533–3551.

[13] M. Ur Rehman, H. Ahmadi, and W. Ul Hassan, "Flash: A comprehensive approach to intrusion detection via provenance graph representation learning," in *Proc. IEEE Symp. Secur. Privacy*, Los Alamitos, CA, USA, May 2024, pp. 3552–3570.

[14] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, "MAGIC: Detecting advanced persistent threats via masked graph representation learning," in *Proc. 33rd USENIX Secur. Symp.*, Philadelphia, PA, USA, Aug. 2024, pp. 5197–5214.

[15] S. Li et al., "NODLINK: An online system for fine-grained APT attack detection and investigation," in *Proc. Netw. Distrib. System Secur. Symp.*, 2024.

[16] A. Goyal, G. Wang, and A. Bates, "R-CAID: Embedding root cause analysis within provenance-based intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 3515–3532.

[17] T. Li et al., "Chaos: Robust spatio-temporal fusion for generalizable APT provenance tracing," in *Proc. Int. Conf. Data Secur. Privacy Protection*, 2025, vol. 16176, pp. 507–524.

[18] D. I2O, "Transparent computing engagement 3," 2018. [Online]. Available: https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md

[19] FiveDirections, "Darpa optc," 2018. [Online]. Available: https://github.com/FiveDirections/OpTC-data

[20] D. Marshall, "Winodws event tracing," 2021. [Online]. Available: https://learn.microsoft.com/en-us/windows-hardware/drivers/devtest/event-tracing-for-windows--etw-

[21] T. Pasquier et al., "Practical whole-system provenance capture," in *Proc. Symp. Cloud Comput.*, 2017, pp. 405–418.

[22] M. N. Hossain, S. Sheikhi, and R. Sekar, "Combating dependence explosion in forensic analysis using alternative tag propagation semantics," in *Proc. IEEE Symp. Secur. Privacy*, 2020, pp. 1139–1155.

[23] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 1137–1152.

[24] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3431–3440.

[25] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg, "Feature hashing for large scale multitask learning," in *Proc. 26th Annu. Int. Conf. Mach. Learn.*, 2009, pp. 1113–1120.

[26] A. H. Khasahmadi, K. Hassani, P. Moradi, L. Lee, and Q. Morris, "Memory-based graph networks," in *Proc. Int. Conf. Learn. Representations*, 2020.

[27] A. Santoro et al., "Relational recurrent neural networks," in *Proc. Adv. Neural Inf. Proc. Syst.*, 2018, pp. 7310–7321.

[28] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Proc. Syst.*, 2017, pp. 6000–6010.

[29] A. Papachristodoulou, C. Kyrkou, and T. Theocharides, "DriveGuard: Robustification of automated driving systems with deep spatio-temporal convolutional autoencoder," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2021, pp. 107–116.

[30] W. U. Hassan et al., "NoDoze: Combatting threat alert fatigue with automated provenance triage," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2019.

[31] Y. Liu et al., "Towards a timely causality analysis for enterprise security," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2018.

[32] M. Defender, "Microsoft defender for endpoint," 2025. [Online]. Available: https://learn.microsoft.com/en-us/defender-endpoint/microsoft-defender-endpoint

[33] K. Church and W. Gale, "Inverse document frequency (IDF): A measure of deviations from poisson," in *Natural Language Processing Using Very Large Corpora*. Berlin, Germany: Springer, 1999, pp. 283–295.

[34] B. Iglewicz and D. C. Hoaglin, *How to Detect and Handle Outliers*, vol. 16 Seattle, WA, USA: Qual. Press, 1993.
[35] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," *J. Amer. Stat. Assoc.*, vol. 88, no. 424, pp. 1273–1283, 1993.
[36] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *Proc. ICLR Workshop Representation Learn. Graphs Manifolds*, 2019.
[37] J. Liu, A. Inam, A. Goyal, K. Westfall, A. Riddle, and A. Bates, "Reapr: Recovery every attack process," 2023. [Online]. Available: https://bitbucket.org/sts-lab/reapr-ground-truth
[38] A. Goyal, X. Han, G. Wang, and A. Bates, "Sometimes, you aren't what you do: Mimicry attacks against provenance graph host intrusion detection systems," in *Proc. 30th Annu. Netw. Distrib. System Secur. Symp.*, 2023.
[39] D. Wagner and P. Soto, "Mimicry attacks on host-based intrusion detection systems," in *Proc. 9th ACM Conf. Comput. Commun. Secur.*, 2002, pp. 255–264.
[40] A. Ahmad, S. Lee, and M. Peinado, "HARDLOG: Practical tamper-proof system auditing using a novel audit device," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 1791–1807.
[41] V. T. Hoang, C. Wu, and X. Yuan, "Faster yet safer: Logging system via Fixed-Key blockcipher," in *Proc. 31st USENIX Secur. Symp.*, Boston, MA, USA, Aug. 2022, pp. 2389–2406.
[42] R. Sekar, H. Kimm, and R. Aich, "eAudit: A fast, scalable and deployable audit data collection system," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 3571–3589.
[43] J. Zeng et al., "SHADEWATCHER: Recommendation-guided cyber threat analysis using system audit records," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 489–506.
[44] F. Dong et al., "DISTDET: A cost-effective distributed cyber threat detection system," in *Proc. 32nd USENIX Secur. Symp.*, Anaheim, CA, USA, Aug. 2023, pp. 6575–6592.
[45] L. Wang et al., "Incorporating gradients to rules: Towards lightweight, adaptive provenance-based intrusion detection," in *Proc. Netw. Distrib. System Secur. Symp.*, 2025.
[46] W. Qiao et al., "Slot: Provenance-driven apt detection through graph reinforcement learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2025, pp. 963–977.
[47] W. Wu et al., "Brewing vodka: Distilling pure knowledge for lightweight threat detection in audit logs," in *Proc. ACM Web Conf.*, New York, NY, USA:Association for Computing Machinery, 2025, pp. 2172–2182, doi: 10.1145/3696410.3714563.
[48] B. Jiang et al., "ORTHRUS: Achieving high quality of attribution in provenance-based intrusion detection systems," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 7173–7192.
[49] T. Bilot et al., "Sometimes simpler is better: A comprehensive analysis of {State-of-the-Art }{Provenance-Based}intrusion detection systems," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 7193–7212.
[50] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.
[51] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, "Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1777–1794.
[52] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2019, pp. 807–817.
[53] B. Xia, J. Yin, J. Xu, and Y. Li, "LogGAN: A sequence-based generative adversarial network for anomaly detection based on system logs," in *Proc. 2nd Int. Conf. Sci. Cyber Secur.*, Nanjing, China, 2019, pp. 61–76.
[54] T. Li et al., "STGraph: Spatio-temporal graph mining for anomaly detection in distributed system logs," in *Proc. 28th Int. Symp. Res. Attacks Intrusions Defenses*, 2025, pp. 190–203.
[55] T. Ngo, J. Yin, Y.-F. Ge, and H. Wang, "Optimizing IoT intrusion detection—A graph neural network approach with attribute-based graph construction," *Information*, vol. 16, no. 6, 2025, Art. no. 499. [Online]. Available: https://www.mdpi.com/2078-2489/16/6/499
[56] W. Hong et al., "A graph empowered insider threat detection framework based on daily activities," *ISA Trans.*, vol. 141, pp. 84–92, 2023.
[57] Y. Gao, H. Yang, P. Zhang, C. Zhou, and Y. Hu, "Graph neural architecture search," in *Proc. Int. Joint Conf. Artif. Intell. Int. Joint Conf. Artif. Intell.*, 2020, pp. 1403–1409.
[58] K. Zhou, X. Huang, Q. Song, R. Chen, and X. Hu, "Auto-GNN: Neural architecture search of graph neural networks," *Front. Big Data*, vol. 5, 2022, Art. no. 1029307.
[59] S. Wang, J. Yin, J. Cao, M. Tang, H. Wang, and Y. Zhang, "ABG-NAS: Adaptive Bayesian genetic neural architecture search for graph representation learning," *Knowl.-Based Syst.*, vol. 328, 2025, Art. no. 114235.
[60] J. Zeng, Z. L. Chua, Y. Chen, K. Ji, Z. Liang, and J. Mao, "WATSON: Abstracting behaviors from audit logs via aggregation of contextual semantics," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2021.
[61] Z. Xu, P. Fang, C. Liu, X. Xiao, Y. Wen, and D. Meng, "DEPCOMM: Graph summarization on system audit logs for attack investigation," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 540–557.
[62] P. Fang et al., "Back-propagating system dependency impact for attack investigation," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 2461–2478.
[63] J. Weston, S. Chopra, and A. Bordes, "Memory networks," in *Proc. 3rd Int. Conf. Learn. Representations*, Y. Bengio and Y. LeCun, Eds., 2015.
[64] S. Sukhbaatar et al., "End-to-end memory networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2440–2448.
[65] X. Yang and P. Fan, "Convolutional end-to-end memory networks for multi-hop reasoning," *IEEE Access*, vol. 7, pp. 135268–135276, 2019.
[66] M. A. Alkhawlani, A. Azman, M. T. Abdullah, R. Yaakob, R. A. Kadir, and E. M. Alshari, "Improving convolutional end-to-end memory networks with BERT for question answering," in *Proc. Intell. Syst. Conf.*, 2024, pp. 90–104.
[67] C. Wang, B. Samari, and K. Siddiqi, "Local spectral graph convolution for point set feature learning," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 52–66.
[68] J. Devlin, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.
[69] A. Ariza-Casabona, B. Twardowski, and T. K. Wijaya, "Exploiting graph structured cross-domain representation for multi-domain recommendation," in *Proc. Adv. Inf. Retrieval*, Cham, Switzerland: Springer Nature Switzerland, 2023, pp. 49–65.